

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

ФАКУЛЬТЕТ ТЕХНОЛОГИЙ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА

Отчёт по лабораторной работе №3

«Разработка одностраничного веб-приложения (SPA) с использованием
фреймворка Vue.JS»

по дисциплине «Web-разработка: Frontend»

Семестр 4

Выполнил: студент

Таряник Антон

гр. J3211

Проверил: Добряков Д. И.

Санкт-Петербург
2026

Содержание

1	Задача	2
2	Ход работы	2
2.1	Инициализация проекта и установка зависимостей	2
2.2	Точка входа и конфигурация (main.js)	2
2.3	Управление состоянием через Pinia	3
2.4	Компонентная архитектура	3
2.5	Composables для переиспользуемой логики	3
2.6	Маршрутизация и защита роутов	4
2.7	Работа с внешним API через axios	4
2.8	Использование директив Vue	5
3	Результаты	5
4	Вывод	8

1 Задача

Целью лабораторной работы является миграция ранее разработанного приложения платформы разметки данных «LabelFlow» (выполненного в рамках ЛР1 и ЛР2 на чистом HTML/CSS/JS с моковым API) на фреймворк Vue.JS с соблюдением требований к разработке одностраничных приложений (SPA).

Требования к реализации:

- Подключение и настройка Vue Router для клиентской маршрутизации;
- Реализация взаимодействия с внешним API посредством библиотеки axios;
- Разумное деление интерфейса на переиспользуемые компоненты (демонстрация компонентного подхода);
- Использование composables для вынесения повторяющейся логики в отдельные модули;
- Сохранение функциональности авторизации, поиска с фильтрацией и управления проектами.

2 Ход работы

2.1 Инициализация проекта и установка зависимостей

Проект создан с помощью официального инструмента скаффолдинга Vite:

```
npm create vue@latest
```

В процессе инициализации подключены: Vue Router (маршрутизация), Pinia (управление состоянием). Дополнительно установлены:

```
npm install axios bootstrap
```

2.2 Точка входа и конфигурация (main.js)

Файл src/main.js инициализирует приложение, подключает глобальные стили, роутер, хранилище и настраивает базовый экземпляр axios:

```
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import App from './App.vue'
import router from './router'
import axios from 'axios'

import 'bootstrap/dist/css/bootstrap.min.css'
import 'bootstrap/dist/js/bootstrap.bundle.min.js'
import './assets/variables.css'
import './assets/styles.css'

axios.defaults.baseURL = 'http://localhost:3000'

const app = createApp(App)
app.use(createPinia())
app.use(router)
app.mount('#app')
```

2.3 Управление состоянием через Pinia

Для работы с авторизацией создано хранилище `src/stores/auth.js`, которое инкапсулирует логику входа/выхода и синхронизацию с `localStorage`:

```
import { defineStore } from 'pinia'
import axios from 'axios'

export const useAuthStore = defineStore('auth', {
  state: () => ({
    user: JSON.parse(localStorage.getItem('currentUser')) || null
  }),
  actions: {
    async login(email, password) {
      const { data } = await axios.get(`/users?email=${encodeURIComponent(email)}`)
      if (data.length > 0 && data[0].password === password) {
        this.user = data[0]
        localStorage.setItem('currentUser', JSON.stringify(this.user))
        return true
      }
      return false
    },
    logout() {
      this.user = null
      localStorage.removeItem('currentUser')
    }
  }
})
```

2.4 Компонентная архитектура

Интерфейс разделён на логические компоненты:

- `AppNavbar.vue` / `GuestNavbar.vue` — навигация для авторизованных и гостевых пользователей;
- `ProjectCard.vue` — карточка проекта для повторного использования в списках;
- `SearchFilters.vue` — блок фильтров для страницы поиска;
- Страницы (`views/`): `LoginView.vue`, `DashboardView.vue`, `SearchView.vue`, `AnnotationView` и др.

2.5 Composables для переиспользуемой логики

Логика переключения темы вынесена в `src/composables/useTheme.js`:

```
import { ref, watchEffect } from 'vue'

export function useTheme() {
  const isDark = ref(localStorage.getItem('theme') === 'dark')

  watchEffect(() => {
    document.body.classList.toggle('dark-mode', isDark.value)
    localStorage.setItem('theme', isDark.value ? 'dark' : 'light')
  })

  const toggle = () => isDark.value = !isDark.value
}
```

```
    return { isDark, toggle }  
  }  
}
```

2.6 Маршрутизация и защита роутов

В `src/router/index.js` настроена ленивая загрузка компонентов и навигационные гварды:

```
import { createRouter, createWebHistory } from 'vue-router'  
import { useAuthStore } from '@/stores/auth'  
  
const router = createRouter({  
  history: createWebHistory(),  
  routes: [  
    { path: '/', name: 'landing', component: () => import('@/views/LandingView.vue') },  
    { path: '/login', name: 'login', component: () => import('@/views/LoginView.vue') },  
    { path: '/dashboard', name: 'dashboard', component: () => import('@/views/DashboardView.vue'), meta: { requiresAuth: true } },  
    // ...  
  ]  
})  
  
router.beforeEach((to, from, next) => {  
  const authStore = useAuthStore()  
  const isAuthenticated = !!authStore.user  
  if (to.meta.requiresAuth && !isAuthenticated) next({ name: 'login' })  
  else if ((to.name === 'login' || to.name === 'registration') && isAuthenticated)  
    next({ name: 'dashboard' })  
  else next()  
})  
  
export default router
```

2.7 Работа с внешним API через axios

В компонентах реализованы запросы к моковому серверу. Пример из `SearchView.vue` — загрузка и фильтрация проектов:

```
<script setup>  
import { ref, computed, onMounted } from 'vue'  
import axios from 'axios'  
  
const projects = ref([])  
const searchQuery = ref('')  
const typeFilter = ref('all')  
  
onMounted(async () => {  
  const { data } = await axios.get('/projects')  
  projects.value = data  
})  
  
const filteredProjects = computed(() => {  
  return projects.value.filter(p => {  
    const matchesSearch = p.name.toLowerCase().includes(searchQuery.value.toLowerCase())  
  })  
})
```

```

    const matchesType = typeFilter.value === 'all' || p.type ===
    typeFilter.value
    return matchesSearch && matchesType
  })
})
</script>

```

2.8 Использование директив Vue

В шаблонах применяются реактивные директивы:

- `v-model` — двустороннее связывание полей формы;
- `v-for` — рендеринг списков проектов и задач;
- `v-if/v-else` — условный отображение элементов интерфейса.

3 Результаты

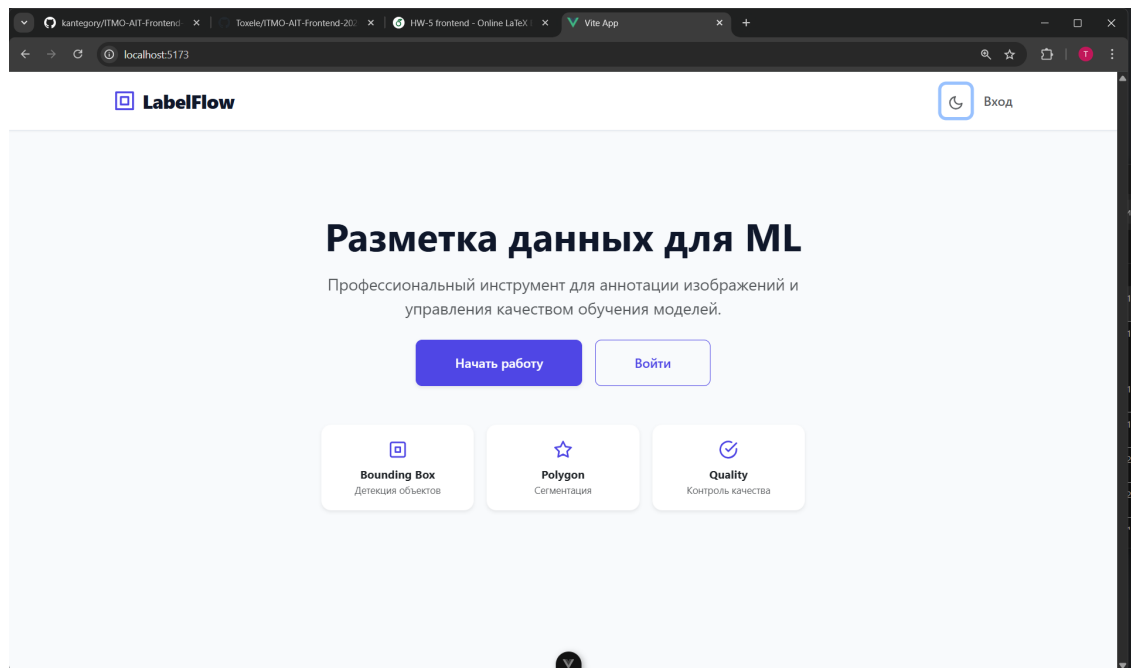


Рис. 1: Страница лендинга приложения (компонент LandingView, светлая тема)

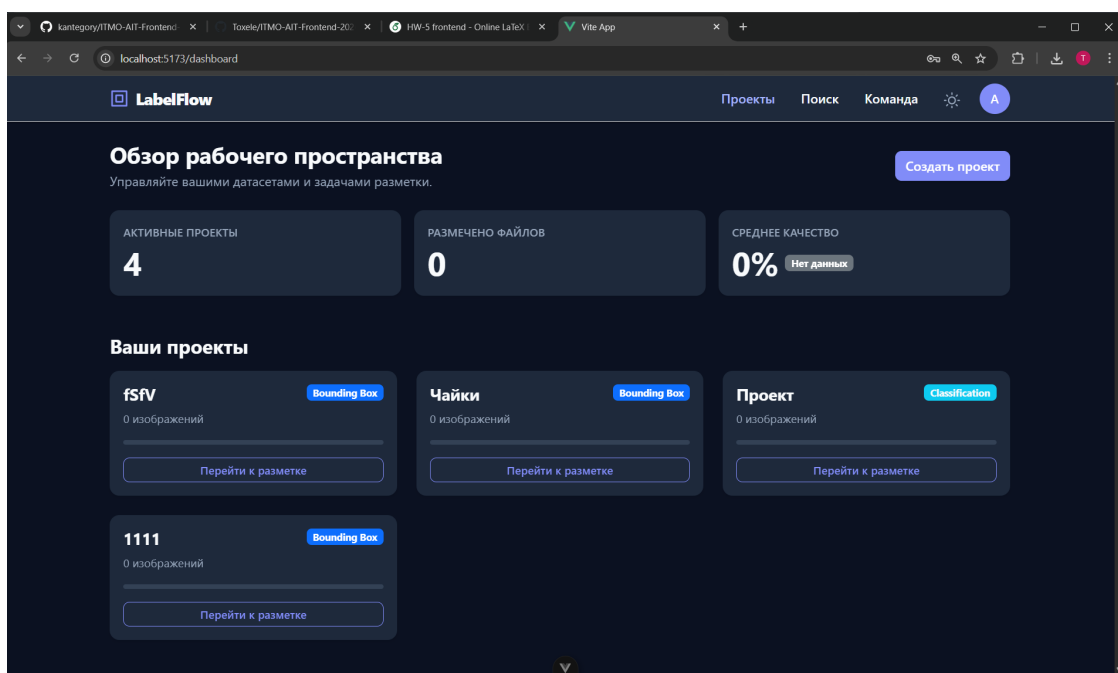


Рис. 2: Панель управления (Dashboard) с отображением проектов пользователя (тёмная тема)

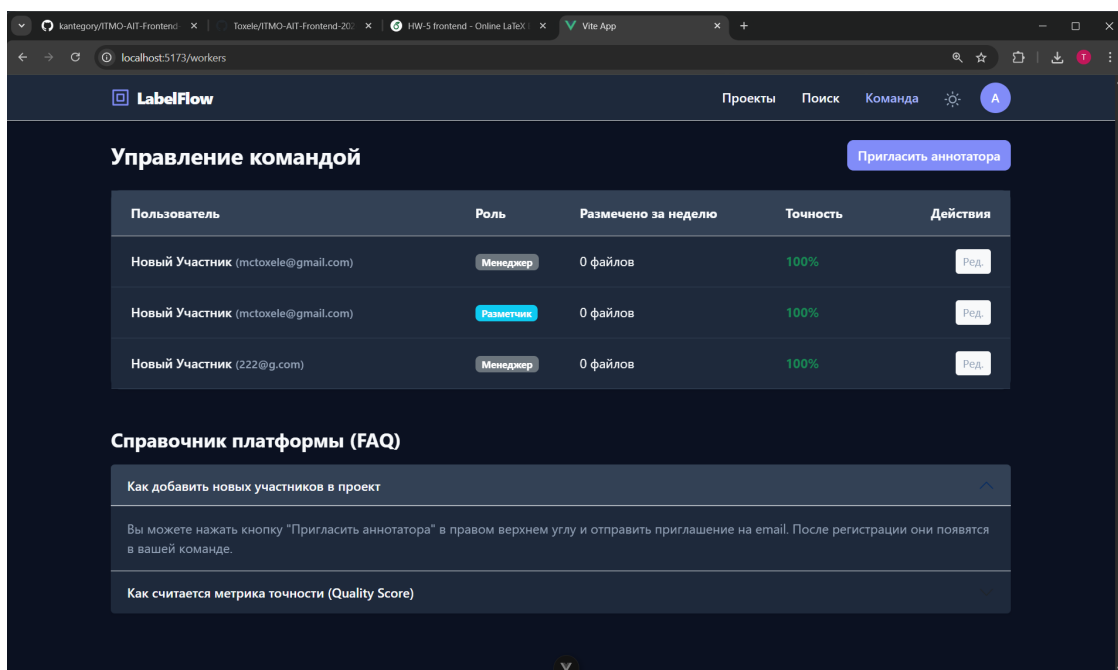


Рис. 3: Страница управления аннотаторами с фильтрацией и поиском

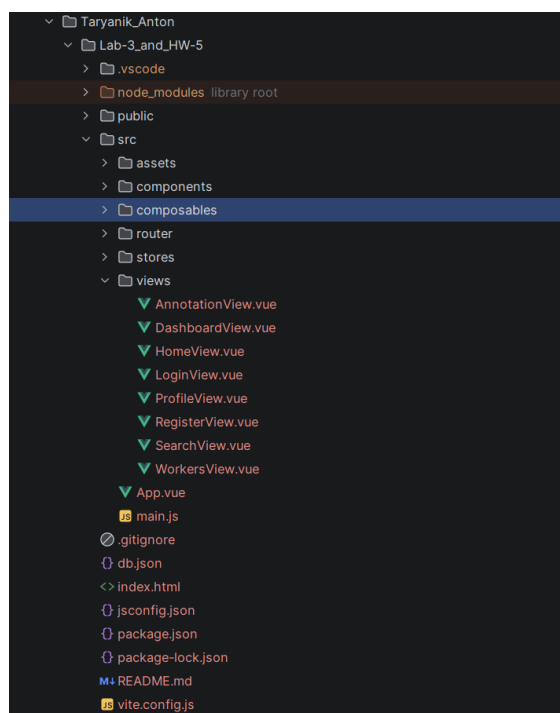


Рис. 4: Структура проекта: разделение на компоненты, представления, хранилища и composables

4 Вывод

В рамках лабораторной работы №3 приложение «LabelFlow» успешно мигрировано на фреймворк Vue.JS с соблюдением всех предъявляемых требований:

- Реализована клиентская маршрутизация с помощью Vue Router, включая защиту маршрутов через навигационные гварды;
- Настроено взаимодействие с внешним (моковым) API через библиотеку axios с обработкой асинхронных запросов;
- Применён компонентный подход: интерфейс разделён на переиспользуемые компоненты, что упрощает поддержку и масштабирование;
- Повторяющаяся логика (переключение темы, валидация форм) вынесена в composables, что соответствует best practices Vue 3;
- Управление состоянием авторизации реализовано через Pinia с сохранением сессии в localStorage.

Приложение функционирует как полноценное SPA: переходы между страницами происходят без перезагрузки, данные подгружаются асинхронно, интерфейс реагирует на действия пользователя в реальном времени. Код структурирован, документирован и готов к дальнейшему расширению функционала.